PROJECT

"Real-time Intrusion Detection and Threat Monitoring Using Suricata IDS, Elastic Stack, and Discord Notification"

Group 8

Authors:

Haikal Raihan Hafidz

Muhammad Salman Mu'aqil

Class: 4CS1

CONTINUING EDUCATION CENTER FOR COMPUTING AND INFORMATION TECHNOLOGY

FACULTY OF ENGINEERING, UNIVERSITY OF INDONESIA 2026

Real-Time Intrusion Detection and Threat Monitoring Using Suricata IDS, Elastic Stack, and Discord Notification

Developed By:

1. Muhammad Salman Muaqil
2. Haikal Raihan Hafiz

Real-Time Intrusion Detection and Threat Monitoring Using Suricata IDS, Elastic Stack, and Discord Notification

Project Title	: Real-Time Intrusion Detection and Threat Monitoring Using Suricata IDS, Elastic Stack, and Telegram Notification
Batch Code	: 4CS1
Start Date	: March 26th, 2026
End Date	: March 26th, 2026
Faculty Name	: Tri Agus Mulyadi
Developers Name	: Haikal Raihan Hafiz Muhammad Salman Muaqil
Submission Date	:

Real-Time Intrusion Detection and Threat Monitoring Using Suricata IDS, Elastic Stack, and Discord Notification

Authors would like to praise to Allah SWT, Most Merciful bless and so authors can finish this Project with the Title is “Real-time Intrusion Detection and Threat Monitoring Using Suricata IDS, Elastic Stack, and Telegram Notification”. Author would like to thanks to Mr.Tri Agus Mulyadi, ST as Lecturer who has given useful suggestion which are help author in writing this paper.

The writer realized that this paper far from perfection, then the writer will be very unfettered the chest and was encouraged, if colleagues and the guide's agreeing lecturer gave the suggestion & criticism for this paper perfection.

I hope this paper can help readers in understanding how threat monitoring configured and works.

.

Depok, March 26th, 2026

Authors

System Analysis

The proposed system aims to provide real-time intrusion detection and threat monitoring by integrating Suricata IDS, Elastic Stack, and Telegram notification services. The need for this system arises from the increasing complexity of cyber threats and the limitations of conventional monitoring approaches, which often rely on manual log analysis and delayed incident response. Existing systems typically lack centralized visibility, automated alerting, and real-time analytics, resulting in slower detection and mitigation of security incidents. Therefore, a unified solution capable of analyzing network traffic in real-time, visualizing threat data, and delivering instant notifications is required.

In this architecture, Suricata functions as the core intrusion detection engine responsible for inspecting network packets and identifying suspicious activities based on predefined rules and signatures. The generated logs are then forwarded to the Elastic Stack, consisting of Elasticsearch, Logstash, and Kibana, to enable centralized log management, data indexing, and visualization. Logstash processes and normalizes Suricata alert data before sending it to Elasticsearch, where the data is stored and indexed for fast retrieval. Kibana provides an interactive dashboard for monitoring detected threats, enabling administrators to analyze traffic patterns, identify anomalies, and evaluate security events efficiently.

To enhance responsiveness, the system integrates Telegram notification services for real-time alert delivery. When Suricata detects potential threats, the processed alerts are automatically forwarded through a notification module that triggers Telegram messages to administrators. This mechanism ensures that security personnel receive immediate warnings without continuously monitoring dashboards. By combining real-time detection, centralized analysis, and instant notification, the proposed system improves situational awareness, reduces response time, and strengthens overall network security monitoring capabilities.

Configuration

Before we jump into the configuration, first we need to install the application via terminal like usual with -sudo apt install elasticsearch-. After the installation, enable the service, start it, and then check the status, if it's running so for this step is done.

```
aerial@DebianServer:~$ sudo systemctl enable elasticsearch
Created symlink '/etc/systemd/system/multi-user.target.wants/elasticsearch.service' → '/usr/lib/systemd/system/elasticsearch.service'.
aerial@DebianServer:~$ sudo systemctl start elasticsearch
aerial@DebianServer:~$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
     Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-03-15 16:41:37 +07; 10s ago
 Invocation: 799855594bf14d4fa4a2ca1d421bd795
       Docs: https://www.elastic.co
   Main PID: 3076 (java)
      Tasks: 101 (limit: 4618)
     Memory: 2.3G (peak: 2.3G)
        CPU: 23.606s
     CGroup: /system.slice/elasticsearch.service
             ├─3076 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=/usr/share/elastic
             ├─3136 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10 -XX:+A
             └─3159 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Mar 15 16:41:24 DebianServer systemd[1]: Starting elasticsearch.service - Elasticsearch...
Mar 15 16:41:26 DebianServer systemd-entrypoint[3076]: WARNING: Unknown module: jdk.internal.vm.ci specified to --add-exports
Mar 15 16:41:26 DebianServer systemd-entrypoint[3076]: WARNING: Unknown module: org.elasticsearch.entitlement.instrumentationjava.logging s
Mar 15 16:41:37 DebianServer systemd[1]: Started elasticsearch.service - Elasticsearch.
lines 1-18/18 (END)
```

Reset the password for the elastic user using the elasticsearch-reset-password command. The generated password will be displayed in the terminal and used later for authentication.

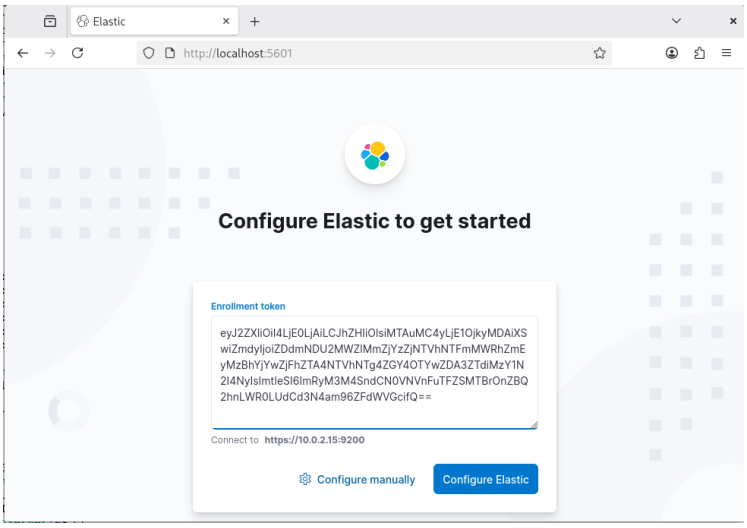
```
aerial@DebianServer:~$ sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
This tool will reset the password of the [elastic] user to an autogenerated value.
The password will be printed in the console.
Please confirm that you would like to continue [y/N]y


Password for the [elastic] user successfully reset.
New value: oGAeEbSR4eYn5oWafeMa
aerial@DebianServer:~$
```

After elastic is done, install Kibana -sudo apt install kibana- and enable the service. Create an enrollment token using the elasticsearch-create-enrollment-token command. This token is required to connect Kibana with Elasticsearch securely.

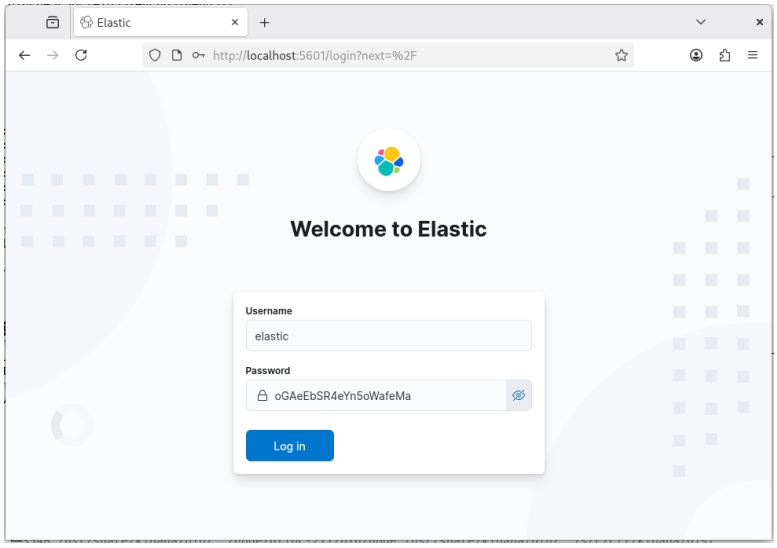
```
aerial@DebianServer:~$ sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMTAuMC4yLjE1OjkyMDAiXSwiZmdyIjoiZDdmNDU2MWZlMmZjYzZjNTVhNTFmMWRhZmEyMzBhYjYwZjFhZTA4NTVhNTg4ZGY4OTYwZDA3ZTdi
MzY1N2I4NyIsImtleSI6ImRyM3M4SndCN0VNVnFuTFZSMTBrOnZBQ2hnLWR0LUdCd3N4am96ZFdWVGcifQ==
aerial@DebianServer:~$
```

Configuration

Open the browser and navigate to http://localhost:5601. Paste the enrollment token and proceed with the configuration.



Use the username elastic and the previously generated password to log in to the Kibana dashboard.



Configuration

Edit the Suricata configuration file, restart the service, enable it to run automatically, and verify that Suricata is active.

```
aerial@DebianServer:~$ sudo nano /etc/suricata/suricata.yaml
aerial@DebianServer:~$ sudo systemctl restart suricata
aerial@DebianServer:~$ sudo systemctl enable suricata
Synchronizing state of suricata.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable suricata
aerial@DebianServer:~$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
     Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset:>
     Active: active (running) since Sun 2026-03-15 17:14:47 +07; 15s ago
 Invocation: a2149246443a4d33bacfb06bf9fee10a
       Docs: man:suricata(8)
             man:suricatasc(8)
             https://suricata.io/documentation/
   Main PID: 2731 (Suricata-Main)
      Tasks: 8 (limit: 4618)
     Memory: 52.3M (peak: 52.4M)
        CPU: 498ms
     CGroup: /system.slice/suricata.service
             └─2731 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.>

Mar 15 17:14:47 DebianServer systemd[1]: Starting suricata.service - Suricata I>
Mar 15 17:14:47 DebianServer suricata[2730]: i: suricata: This is Suricata vers>
Mar 15 17:14:47 DebianServer systemd[1]: Started suricata.service - Suricata ID>
lines 1-17/17 (END)...skipping...
```

Open the Filebeat Suricata module configuration file and enable the Suricata event logs (eve.json).

```
aerial@DebianServer:~$ sudo nano /etc/filebeat/modules.d/suricata.yml

  GNU nano 8.4                                  /etc/filebeat/modules.d/suricata.yml
# Module: suricata
# Docs: https://www.elastic.co/guide/en/beats/filebeat/8.19/filebeat-module-suricata.html

- module: suricata
  # All logs
  eve:
    enabled: false

    # Set custom paths for the log files. If left empty,
    # Filebeat will choose the paths depending on your OS.
    #var.paths: ["/var/log/suricata/eve.json"]
```

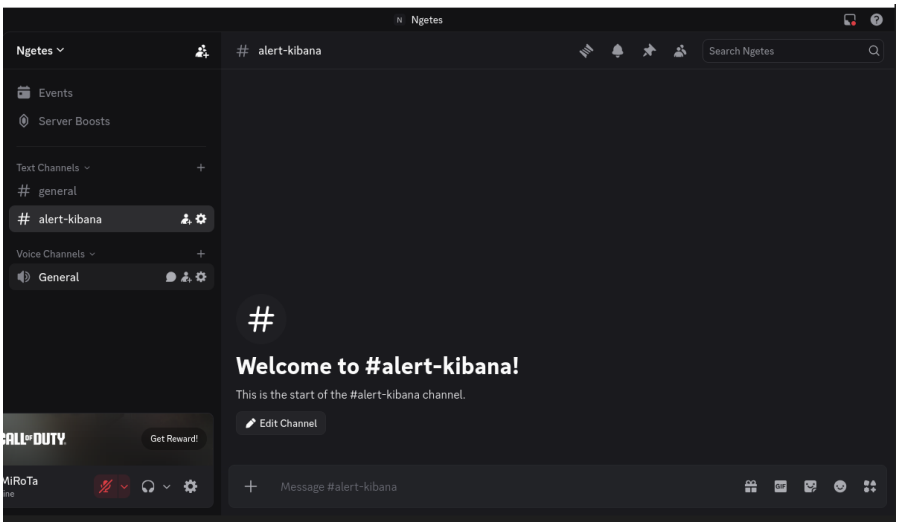
Run filebeat test output to confirm that Filebeat can successfully connect to Elasticsearch.

```
aerial@DebianServer:~$ sudo filebeat test output
elasticsearch: https://localhost:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: ::1, 127.0.0.1
    dial up... OK
  TLS...
    security... WARN server's certificate chain verification is disabled
    handshake... OK
    TLS version: TLSv1.3
    dial up... OK
  talk to server... OK
  version: 8.19.12
aerial@DebianServer:~$ sudo filebeat setup
Overwriting lifecycle policy is disabled. Set `setup.ilm.overwrite: true` to overwrite.
Index setup finished.
Loading dashboards (Kibana must be running and reachable)
Loaded dashboards
Loaded Ingest pipelines
aerial@DebianServer:~$
```

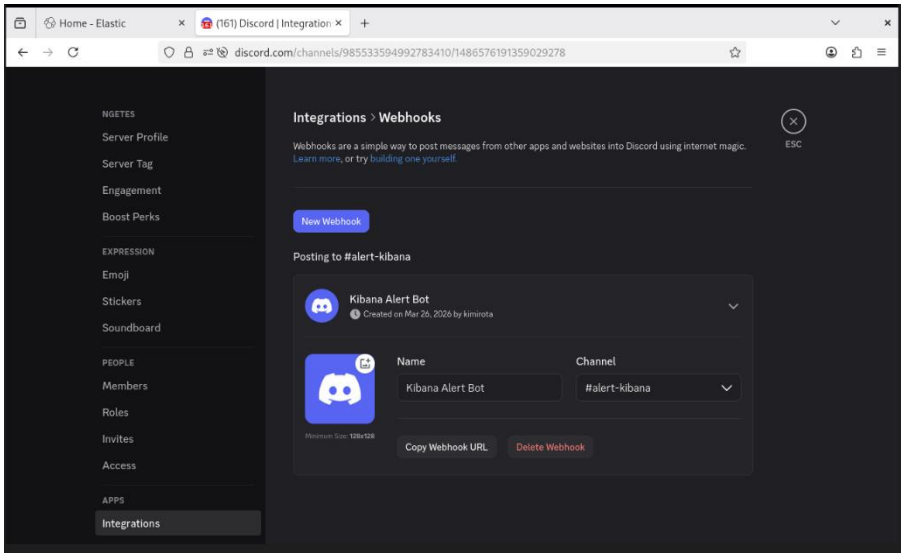
Configuration

Now is the configuration for alrt notification via Discord:

Create a new text channel in Discord named **#alert-kibana**.
This channel will be used to receive alerts generated from Suricata and sent via webhook.



Go to **Server Settings → Integrations → Webhooks → New Webhook**, then assign it to the **#alert-kibana** channel and copy the webhook URL.



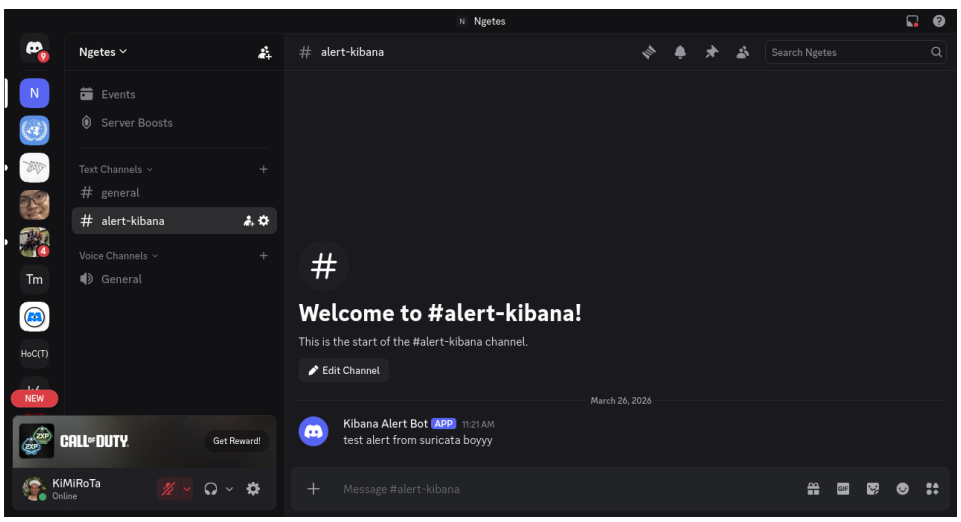
Configuration

Send a test message using curl:

```
aerial@DebianServer:~$ curl -H "Content-Type: application/json" -d '{"content":"test alert from suricata boyyy"}' https://discord.com/api/webhooks/1486575528944078849/EXX2LsAxffQ3wEsP-Vf7NklobnYqCc80WEof_cfeoOdEKKi7iD1DFrL5fZbMyLl5WtQs
```

This step verifies that the webhook works correctly before integrating it into the alert automation script.

Check the **#alert-kibana** channel and confirm the test message appears.



The message is received, the webhook configuration is successful and ready for automation.

Create a shell script named alert.sh.
This script will query Elasticsearch for Suricata alerts and send formatted notifications to Discord using the webhook.

```
aerial@DebianServer:~$ sudo nano alert.sh
[sudo] password for aerial:
```

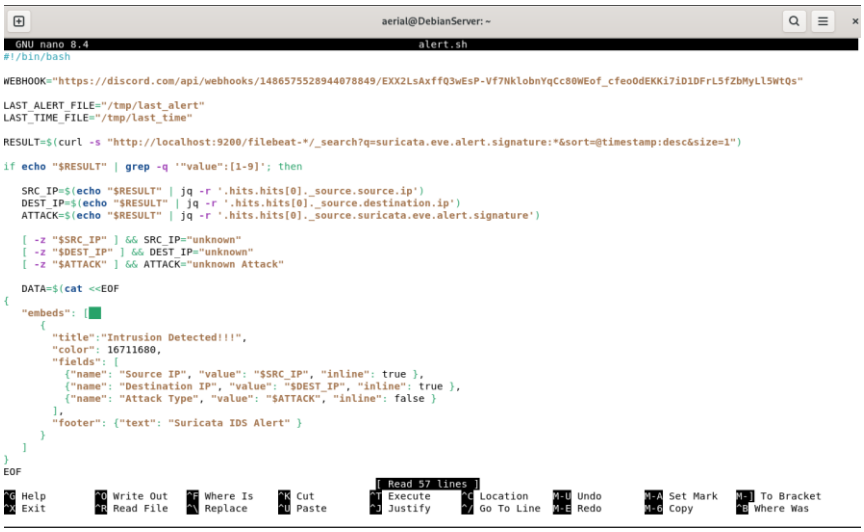
Configuration

Define the webhook URL and temporary files for tracking alerts.
The script queries Elasticsearch for the latest Suricata alert using curl and extracts:

- Source IP
- Destination IP
- Attack signature

The script then prepares a JSON payload using Discord embed format.

Add logic to prevent duplicate alerts.
The script compares the current alert with the previous one and checks the timestamp.
If the alert is new, it sends the notification to Discord and updates the tracking files.



Configuration

The script sends the alert to Discord using:

curl -H "Content-Type: application/json" -d "$DATA" "$WEBHOOK"

This posts a formatted intrusion alert message containing:

- Source IP
- Destination IP
- Attack type
- Suricata IDS footer

```
TIMESTAMP=$(date +%s)
CURRENT_ALERT="$SRC_IP-$DEST_IP-$ATTACK"

   if [ -f $LAST_ALERT_FILE ] && [ -f $LAST_TIME_FILE ]; then
      LAST_ALERT=$(cat $LAST_ALERT_FILE)
      LAST_TIME=$(cat $LAST_TIME_FILE)

      DIFF=$((CURRENT_TIME - LAST_TIME))

      if [ "$CURRENT_ALERT" == "$LAST_ALERT" ] && [ $DIFF -lt 60 ]; then
        exit 0
      fi
   fi

   curl -H "Content-Type: application/json" -d "$DATA" "$WEBHOOK"

   echo "$CURRENT_ALERT" > $LAST_ALERT_FILE
   echo "$CURRENT_TIME" > $LAST_TIME_FILE

fi

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy       ^B Where Was
```

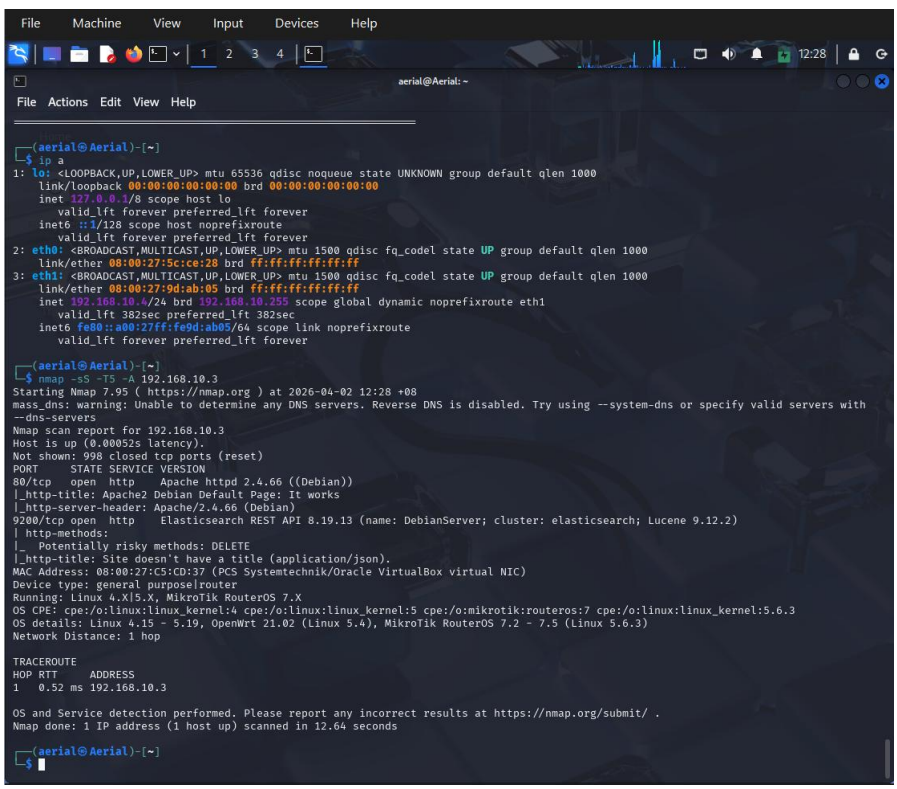
Demonstration

The script sends the alert to Discord using:

curl -H "Content-Type: application/json" -d "$DATA" "$WEBHOOK"

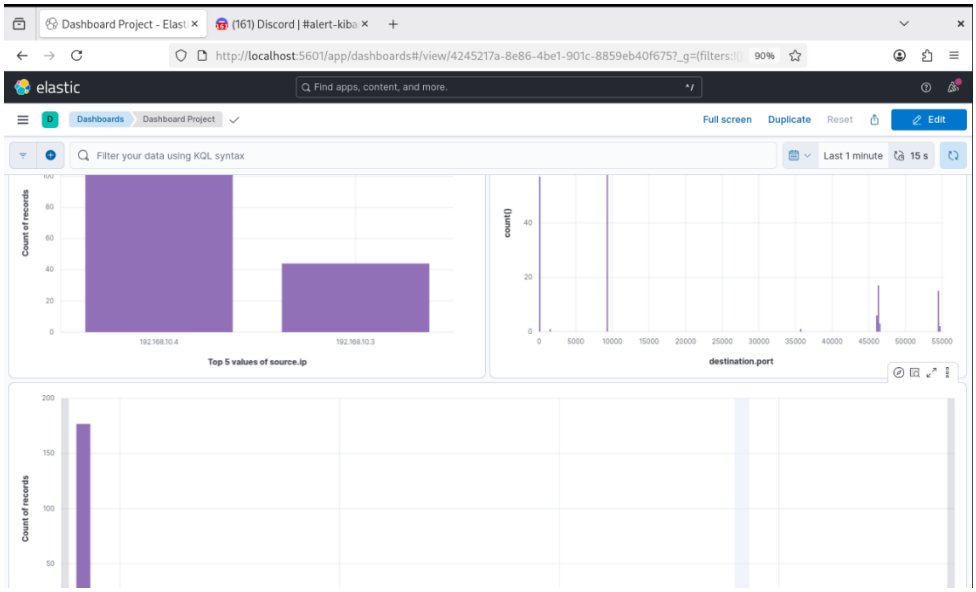
This posts a formatted intrusion alert message containing:

- Source IP
- Destination IP
- Attack type
- Suricata IDS footer

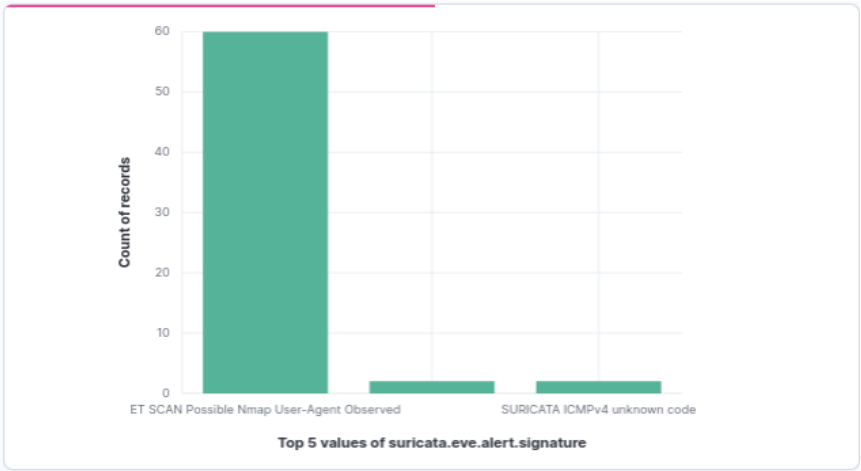


Demonstration

The Kibana dashboard displays incoming Suricata events in real time. Visualizations such as **Top Source IP**, **Destination Ports**, and **Alert Signatures** help identify suspicious activity generated from the scan.



The dashboard shows an alert labeled **"ET SCAN Possible Nmap User-Agent Observed"**. This confirms that Suricata successfully detected the Nmap scanning activity.



Demonstration

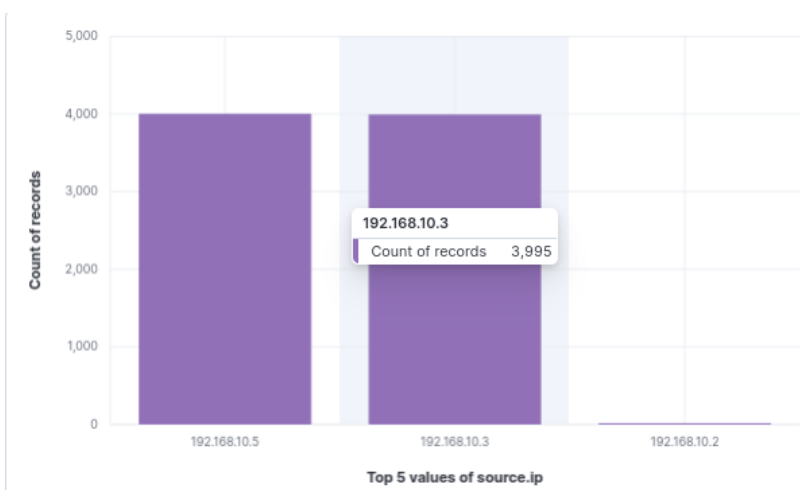
A directory brute-force attack is performed using Gobuster:

-gobuster dir -u http://192.168.10.3 -w /usr/share/wordlists/dirb/common.txt-

This attempts to discover hidden directories and files on the target web server.

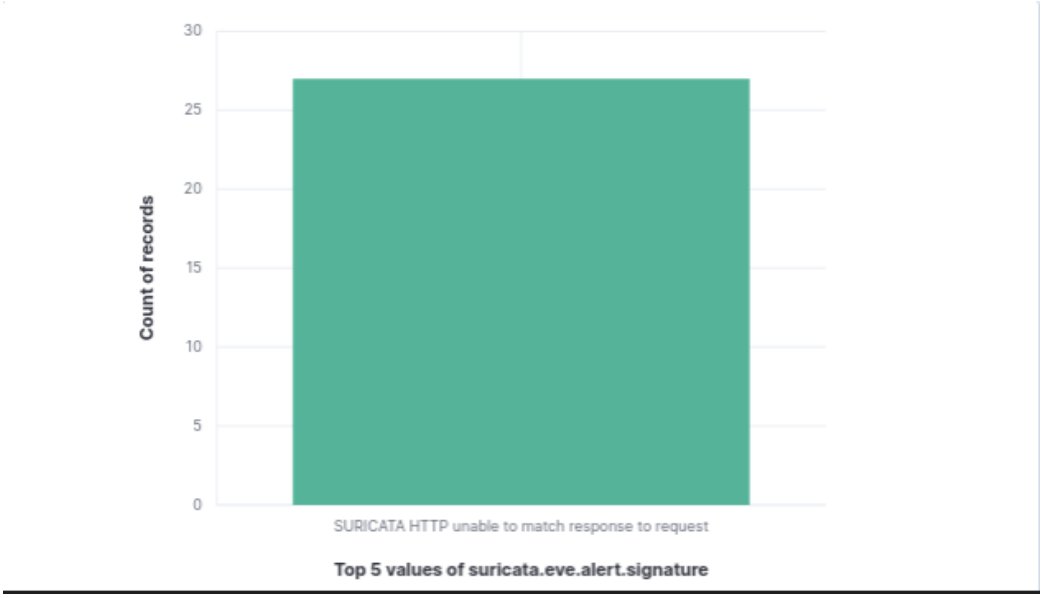
```
┌──(aerial㉿Aerial)-[~]
└─$ gobuster dir -u http://192.168.10.3 -w /usr/share/wordlists/dirb/common.txt
═══════════════════════════════════════════════════════════════════
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════════
[+] Url:                     http://192.168.10.3
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════════
.htaccess            (Status: 403) [Size: 317]
.hta                 (Status: 403) [Size: 317]
.htpasswd            (Status: 403) [Size: 317]
index.html           (Status: 200) [Size: 10703]
server-status        (Status: 403) [Size: 317]
Progress: 4613 / 4613 (100.00%)
═══════════════════════════════════════════════════════════════════
Finished
═══════════════════════════════════════════════════════════════════
```

After running Gobuster, the Kibana dashboard shows a spike in event counts. The source IP generating the highest number of alerts is identified, indicating suspicious enumeration behavior.



Demonstration

Suricata generates alerts such as:
"SURICATA HTTP unable to match response to request"
This indicates abnormal HTTP requests caused by automated scanning tools.

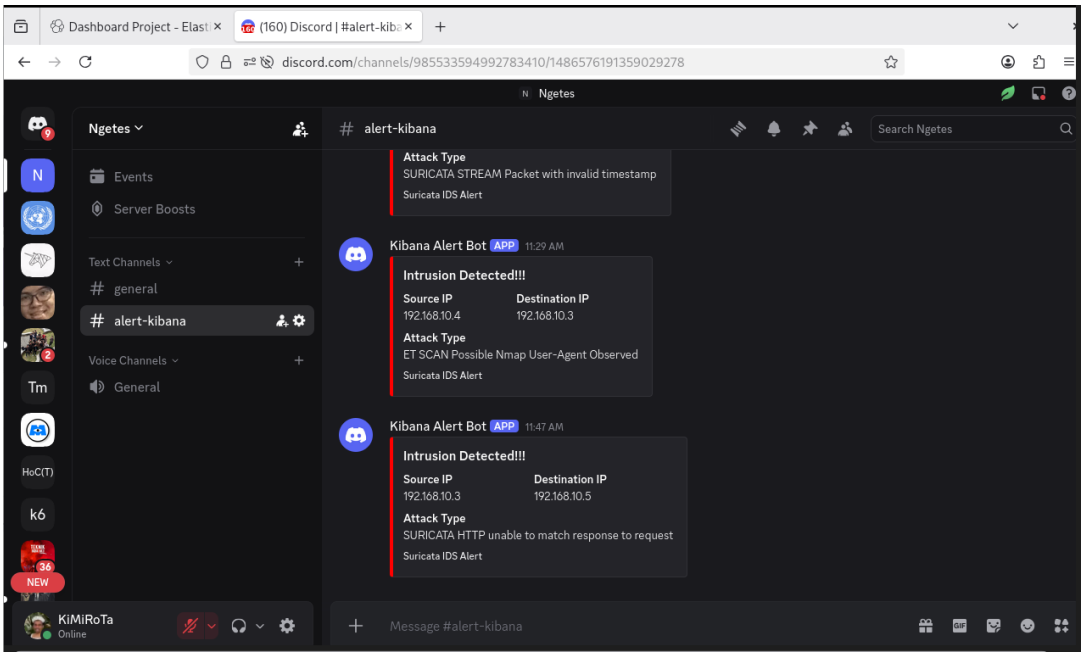


Demonstration

The configured alert script automatically sends intrusion notifications to Discord. The alert message includes:

- Source IP
- Destination IP
- Attack type
- Suricata IDS label

This demonstrates successful end-to-end integration:
Attack → Suricata Detection → Elasticsearch → Kibana → Discord Alert



Suggestiom and Improvement

For future improvements, several enhancements can be considered to make the system more robust and scalable. First, additional Suricata rules and threat intelligence feeds can be integrated to improve detection accuracy and coverage of emerging threats. Second, role-based access control and authentication mechanisms can be implemented in Kibana to improve dashboard security.

The notification system can be expanded by integrating multiple alert channels such as email or Telegram to ensure redundancy. Machine learning-based anomaly detection could also be explored to identify unknown threats beyond signature-based detection. Lastly, deploying the system in a distributed environment would improve scalability and allow monitoring of larger network infrastructures.

The proposed intrusion detection system can be further enhanced by implementing automated response capabilities to transform it into an Intrusion Detection and Prevention System (IDPS). Integrating firewall mechanisms such as iptables would allow the system to automatically block malicious IP addresses in real time, reducing response delays. In addition, applying severity-based alert classification would enable different actions depending on threat levels, where high-risk attacks trigger immediate blocking while lower-risk events generate notifications only. Alert enrichment can also be improved by including contextual information such as geolocation, attack frequency, and historical activity, helping administrators better understand incidents and respond more effectively.

Further improvements can focus on dashboard optimization and system scalability. Enhancing Kibana visualizations with threat trends, severity indicators, and real-time monitoring views would improve situational awareness. Expanding attack simulations to include various threats such as brute-force attacks, web application attacks, and advanced techniques would increase detection reliability. The notification system can also be extended to multiple platforms to ensure alert redundancy. Additionally, implementing log normalization mechanisms would improve data consistency, especially in cases where source and destination IP addresses appear reversed, ensuring more accurate analysis and reporting.

Device Specifocation and Tools

PC: ROG Strix G16

Tools:

- Oracle Virtual Box (Kali Linux, Debian)
- Kibana
- Elasticsearch
- Suricata
- Discord
- Filebeat
- Nmap
- gobuster
- hping3